

07. Diffusion Theory

Overview

Diffusion models define a forward noising process and learn to reverse it. This module derives the variance schedule, closed-form noisy samples, and the connection between score matching and denoising objectives. You need this foundation before DDPM sampling and latent diffusion in the next module.

Lab: Lab 07.

Prior modules: Module 05.

Contents

1	From GANs to Diffusion	2
2	Forward Diffusion Process	2
3	Reverse Process Goal	2
4	Variational Lower Bound	2
5	Noise Schedules	2
6	Score Matching Connection	3
7	Lab	3
8	Signal-to-Noise Ratio	3
8.1	Stochastic vs. Deterministic Sampling	3
9	Extended Intuition	3
9.1	Score Matching Connection	4
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	5
13	Debugging Checklist	5
14	Practice Problems	5

Module track

This module builds on **Module 05**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

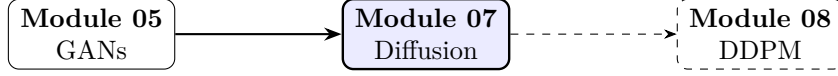


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 From GANs to Diffusion

While GANs (**Module 05** (GAN objective)) learn implicit samplers, **diffusion models** define a forward noising process and learn its reverse. Foundations support DDPM [2] in **Module 08** (DDPM loss).

2 Forward Diffusion Process

Define Markov chain adding Gaussian noise over $t = 1, \dots, T$:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (1)$$

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad (2)$$

where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

Note. The closed form $q(\mathbf{x}_t | \mathbf{x}_0)$ enables single-step noising during training, no sequential simulation required.

3 Reverse Process Goal

We learn $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ to denoise. For small β_t , reverse is also Gaussian:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)). \quad (3)$$

4 Variational Lower Bound

Maximize ELBO on $\mathbf{x}_0$:

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] = \mathcal{L}_{\text{VLB}}. \quad (4)$$

Decomposes into reconstruction, prior matching, and denoising terms across timesteps.

5 Noise Schedules

Linear: β_t from 10^{-4} to 0.02 over $T = 1000$. Cosine schedule improves low-noise regions:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2 \left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2} \right). \quad (5)$$

Table 1: Noise schedule comparison.

Schedule	Simplicity	Sample quality
Linear	High	Good
Cosine	Medium	Better (low SNR)
Learned	Low	Research frontier

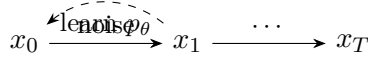


Figure 2: Forward noising (solid) and learned reverse (dashed).

6 Score Matching Connection

The score $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ relates to predicted noise ϵ_θ :

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}, \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (6)$$

7 Lab

Lab 07 visualizes $q(\mathbf{x}_t | \mathbf{x}_0)$ from (1) across t . Implement DDPM training objective in **Module 08** (DDPM loss).

8 Signal-to-Noise Ratio

Define $\text{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$; denoising difficulty increases with t . Predicting $\mathbf{x}_0$ directly relates to noise via Tweedie’s formula:

$$\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t] = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon \right). \quad (7)$$

8.1 Stochastic vs. Deterministic Sampling

DDIM skips substeps using non-Markovian inference for faster generation.

9 Extended Intuition

Diffusion models define a forward process that gradually adds Gaussian noise until data becomes nearly pure noise. The reverse process learns to denoise step by step, recovering structure from $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$. Instead of adversarial balance, training reduces to supervised noise prediction (or score matching) at random timesteps.

The variance schedule β_t controls how much noise enters per step. Small β_t over many steps ($T = 1000$) yields a smooth corruption path. Ho et al. use linear schedule from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$; cosine schedules often improve low-noise steps.

Closed-form sampling of $\mathbf{x}_t$ given $\mathbf{x}_0$ avoids iterating the forward chain during training:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}\right), \quad (8)$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. The network typically predicts noise ϵ that was used to construct $\mathbf{x}_t$, not $\mathbf{x}_0$ directly. DDPM in **Module 08** turns this into a trainable sampler with discrete reverse updates.

9.1 Score Matching Connection

The score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ points toward higher density regions. Denoising score matching trains a network to predict noise ϵ which is proportional to the score under Gaussian corruption. Signal-to-noise ratio $\text{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$ drops exponentially in t for linear schedules; low-SNR steps are hardest to learn. Training uniformly over t oversamples easy high-noise steps relative to perceptual importance; some schedules reweight loss by SNR or use cosine β_t to balance. Conditional diffusion adds class or text embedding to ϵ_θ input; classifier-free guidance in **Module 08** builds on the same dropout trick at train time. Forward process is Markov: $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ depends only on the previous step, enabling single-step training samples at any t .

10 Numerical Walkthrough

1D toy for intuition: $\mathbf{x}_0 = [2.0]$, $\beta_t = 0.1$, so $\alpha_t = 0.9$, $\bar{\alpha}_1 = 0.9$.

Noisy sample: $\mathbf{x}_1 = \sqrt{0.9} \cdot 2.0 + \sqrt{0.1} \epsilon = 1.897 + 0.316\epsilon$. If $\epsilon = 0.5$, $\mathbf{x}_1 \approx 2.05$ (still close to signal).

After $t = 10$ with constant $\beta = 0.1$: $\bar{\alpha}_{10} = 0.9^{10} \approx 0.349$, signal scale $\sqrt{\bar{\alpha}_{10}} \approx 0.59$. After $t = 50$: $\bar{\alpha}_{50} \approx 0.005$, mean coefficient on $\mathbf{x}_0$ is $\sqrt{0.005} \approx 0.07$: mostly noise.

Training draw: pick $t \sim \text{Uniform}\{1, \dots, T\}$, sample $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, build $\mathbf{x}_t$, regress $\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2$. For MNIST $\mathbf{x}_0 \in \mathbb{R}^{784}$, $T = 1000$, $B = 64$, a small MLP noise predictor might need 200k steps to produce blurry digits before conv models help.

At $t = 500$ with linear schedule to $\beta_T = 0.02$: $\bar{\alpha}_{500} \approx \exp(-\sum \beta) \ll 1$; the network must infer structure from mostly noise. Predicting $\mathbf{x}_0$ directly is equivalent algebraically to predicting ϵ , but ϵ targets have unit variance at every t , which stabilizes MSE loss scaling. Always sample a fresh $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ each training step; reusing noise couples batches incorrectly.

Forward diffusion visualization: plot $\mathbf{x}_t$ at $t \in \{0, 100, 500, 999\}$ for one MNIST digit; human eye loses structure around t where $\sqrt{\bar{\alpha}_t} < 0.3$. Reverse sampling is not trained explicitly; quality depends entirely on whether ϵ_θ generalizes across all t , especially mid-range SNR.

11 PyTorch Implementation Notes

- Precompute $\bar{\alpha}_t$, $\sqrt{\bar{\alpha}_t}$, $\sqrt{1 - \bar{\alpha}_t}$ buffers; register with `register_buffer` so they move with device.
- Timestep t embedding: sinusoidal like Transformers, or `nn.Embedding(T, dim)` indexed by integer t .
- Training noise: `x_t = sqrt_ab[t]*x0 + sqrt_1m_ab[t]*eps` with broadcast shapes `(B,1,1,1)` for images.
- Loss: `F.mse_loss(eps, model(x_t, t))`; optional weighting by signal-to-noise ratio at advanced stage.
- Do not backprop through the forward diffusion sampling loop during training; single-step prediction only.

```
t = torch.randint(0, T, (B,), device=x0.device)
eps = torch.randn_like(x0)
x_t = extract(sqrt_alphas_cumprod, t, x0.shape) * x0 \
      + extract(sqrt_one_minus_alphas_cumprod, t, x0.shape) * eps
loss = F.mse_loss(model(x_t, t), eps)
```

12 Local GPU Constraints

Theory module math runs on CPU; preview training a tiny MLP denoiser on flattened MNIST fits in 4 GB easily. Store schedule tensors on GPU once; avoid recomputing $\bar{\alpha}_t$ each batch.

When jumping to conv U-Net in **Module 08**, 28×28 , $B = 128$ is feasible; 64×64 needs $B \leq 32$ on a 1650. Mixed precision safe for MSE noise prediction in most setups.

Cosine schedule (Nichol & Dhariwal): $\bar{\alpha}_t = \cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right)$ with small s prevents β_t from blowing up at $t = 0$. ELBO connection: DDPM training approximates variational bound on log-likelihood; simplified MSE drops weighting terms that matter at low t but suffice for MNIST labs. Score-based SDE view: forward SDE adds noise continuously; reverse SDE uses learned score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$; DDPM is a discretization (continuous theory in **Module 08** sampling). Variance-preserving (VP) vs variance-exploding (VE) schedules change how $\bar{\alpha}_t$ behaves at large t ; Ho DDPM is VP; know which your code implements.

13 Debugging Checklist

- Loss decreases but samples are noise: reverse sampler not implemented or wrong $\bar{\alpha}_t$ indexing off-by-one.
- Exploding $\mathbf{x}_t$ values: forgot to normalize images to $[-1, 1]$ or unit variance before noising.
- Timestep embedding ignored: model output independent of t ; verify embedding concatenation or AdaGN injection.
- Schedule too aggressive: β_T too large makes $\bar{\alpha}_T \approx 0$ too early; try cosine schedule.
- Shape bug on broadcast: `t` shape `(B,)` must expand to image dims via helper `extract`.
- Loss flat at 1.0: model ignoring t embedding; verify sinusoidal or learned time index is concatenated into hidden state.

14 Practice Problems

Problem 1. With $\beta_t = 0.02$ constant, compute $\bar{\alpha}_{100}$ and interpret signal fraction $\sqrt{\bar{\alpha}_{100}}$.

Problem 2. Derive $\mathbf{x}_t$ as linear combo of $\mathbf{x}_0$ and ϵ given the Gaussian forward kernel.

Problem 3. Plot $\sqrt{\bar{\alpha}_t}$ vs t for linear vs cosine schedules on the same plot.

Problem 4. Train a scalar denoiser on 1D mixture of Gaussians and visualize reverse samples at $t = 100, 500, 999$.

Problem 5. For fixed $\mathbf{x}_0$ and ϵ , plot $\|\mathbf{x}_t - \mathbf{x}_0\|_2$ vs t . Compare linear vs cosine $\bar{\alpha}_t$ schedules.

Problem 6. Implement cosine β_t schedule and log $\bar{\alpha}_T$. Confirm $\bar{\alpha}_T > 0$ so $q(\mathbf{x}_T | \mathbf{x}_0)$ retains slight signal scale.

Denoising at each t is band-pass filtering: high t removes all frequencies; the network restores structure frequency-by-frequency across training steps. Hybrid loss weighting $\lambda(t)$ by SNR is optional; start with uniform t sampling before tuning per-timestep weights.

Visualize forward noise in **Lab 07**; full DDPM loop in **Module 08**.

References

- [1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, et al. Generative adversarial nets. In *NeurIPS*, 2014.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

-
- [3] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
 - [4] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.
 - [5] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, et al. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.